```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

url = "https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv"

try:
    df = pd.read_csv(url)
    print("Dataset loaded successfully from URL")
except Exception:
    print("Upload the Telco Customer Churn CSV file")
    uploaded = files.upload()
    df = pd.read_csv(next(iter(uploaded)))
    print("Dataset loaded successfully")
```

```
Dataset loaded successfully from URL
```

```
print("First five records:")
display(df.head())

print("Last five records:")
display(df.tail())
```

```
First five records:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecuri
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

5 rows × 21 columns

```
Last five records:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSe
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL	
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fiber optic	
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL	
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fiber optic	
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fiber optic	

5 rows × 21 columns

```
print("Dataset shape:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nDataset information:")
df.info()

print("\nData types:")
print(df.dtypes)
```

```
Dataset shape: (7043, 21)

Column names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetSer

Dataset information:
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   customerID        7043 non-null   object 
 1   gender            7043 non-null   object 
 2   SeniorCitizen     7043 non-null   int64  
 3   Partner           7043 non-null   object 
 4   Dependents        7043 non-null   object 
 5   tenure            7043 non-null   int64  
 6   PhoneService      7043 non-null   object 
 7   MultipleLines     7043 non-null   object 
 8   InternetService   7043 non-null   object 
 9   OnlineSecurity    7043 non-null   object 
 10  OnlineBackup      7043 non-null   object 
 11  DeviceProtection  7043 non-null   object 
 12  TechSupport       7043 non-null   object 
 13  StreamingTV       7043 non-null   object 
 14  StreamingMovies   7043 non-null   object 
 15  Contract          7043 non-null   object 
 16  PaperlessBilling  7043 non-null   object 
 17  PaymentMethod     7043 non-null   object 
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object 
 20  Churn             7043 non-null   object 
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB

Data types:
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
```

```
print("Missing values in each column:")
print(df.isnull().sum())

print("\nTotal missing values:", df.isnull().sum().sum())

print("\nDuplicate records:", df.duplicated().sum())
```

```
Missing values in each column:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Total missing values: 0

Duplicate records: 0
```

```
print("Summary statistics:")
display(df.describe(include="all").T)
```

```
Summary statistics:
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
customerID	7043	7043	3186-AJIEK	1	NaN	NaN	NaN	NaN	NaN	NaN	NaN
gender	7043	2	Male	3555	NaN	NaN	NaN	NaN	NaN	NaN	NaN
SeniorCitizen	7043.0	NaN	NaN	NaN	0.162147	0.368612	0.0	0.0	0.0	0.0	1.0
Partner	7043	2	No	3641	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Dependents	7043	2	No	4933	NaN	NaN	NaN	NaN	NaN	NaN	NaN
tenure	7043.0	NaN	NaN	NaN	32.371149	24.559481	0.0	9.0	29.0	55.0	72.0
PhoneService	7043	2	Yes	6361	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MultipleLines	7043	3	No	3390	NaN	NaN	NaN	NaN	NaN	NaN	NaN
InternetService	7043	3	Fiber optic	3096	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineSecurity	7043	3	No	3498	NaN	NaN	NaN	NaN	NaN	NaN	NaN
OnlineBackup	7043	3	No	3088	NaN	NaN	NaN	NaN	NaN	NaN	NaN
DeviceProtection	7043	3	No	3095	NaN	NaN	NaN	NaN	NaN	NaN	NaN
TechSupport	7043	3	No	3473	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingTV	7043	3	No	2810	NaN	NaN	NaN	NaN	NaN	NaN	NaN
StreamingMovies	7043	3	No	2785	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Contract	7043	3	Month-to-month	3875	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaperlessBilling	7043	2	Yes	4171	NaN	NaN	NaN	NaN	NaN	NaN	NaN
PaymentMethod	7043	4	Electronic check	2365	NaN	NaN	NaN	NaN	NaN	NaN	NaN
MonthlyCharges	7043.0	NaN	NaN	NaN	64.761692	30.090047	18.25	35.5	70.35	89.85	118.75
TotalCharges	7043	6531		11	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Churn	7043	2	No	5174	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
df = df.drop_duplicates().copy()

df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)

for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        df[col] = df[col].fillna(df[col].median())
    else:
        mode = df[col].mode(dropna=True)
        if len(mode) > 0:
            df[col] = df[col].fillna(mode.iloc[0])

print("Missing values after cleaning:")
print(df.isnull().sum())

print("\nCleaned dataset shape:", df.shape)
display(df.head())
```

```
Missing values after cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

Cleaned dataset shape: (7043, 21)
```

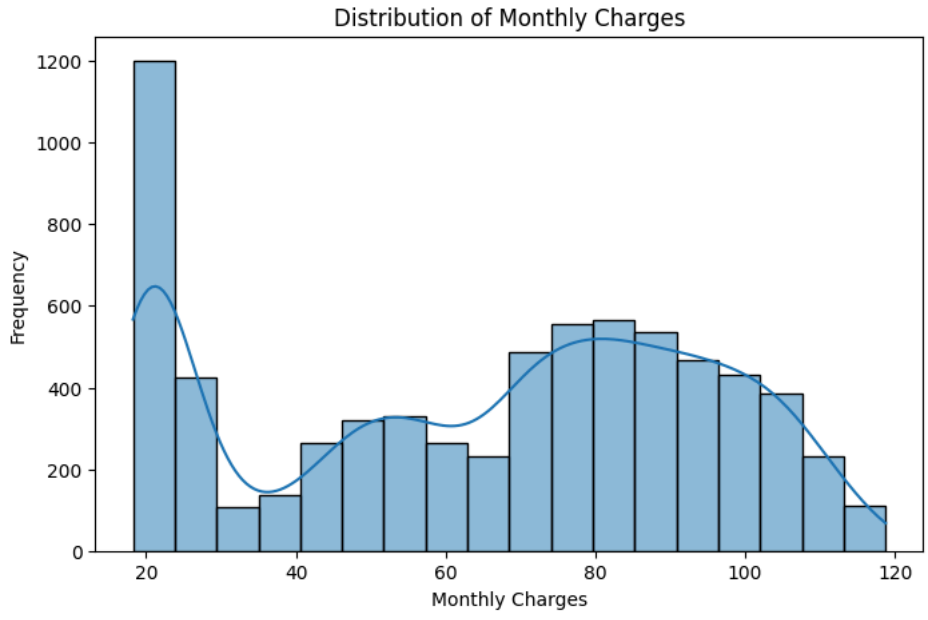
	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecuri
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	

5 rows × 21 columns

```
plt.figure(figsize=(8, 5))

sns.histplot(df["MonthlyCharges"], kde=True)

plt.title("Distribution of Monthly Charges")
plt.xlabel("Monthly Charges")
plt.ylabel("Frequency")
plt.show()
```

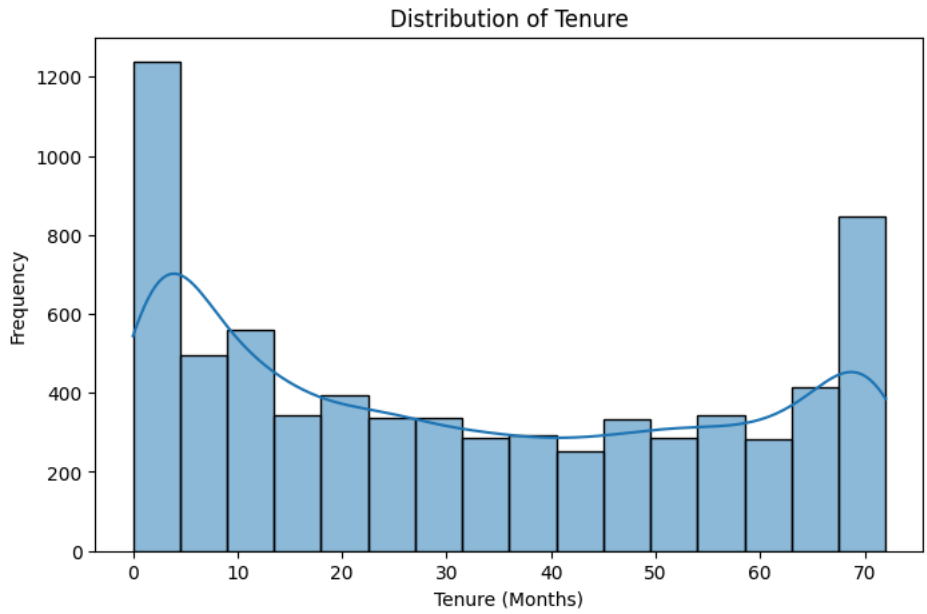



```
plt.figure(figsize=(8, 5))

sns.histplot(df["tenure"], kde=True)

plt.title("Distribution of Tenure")
```

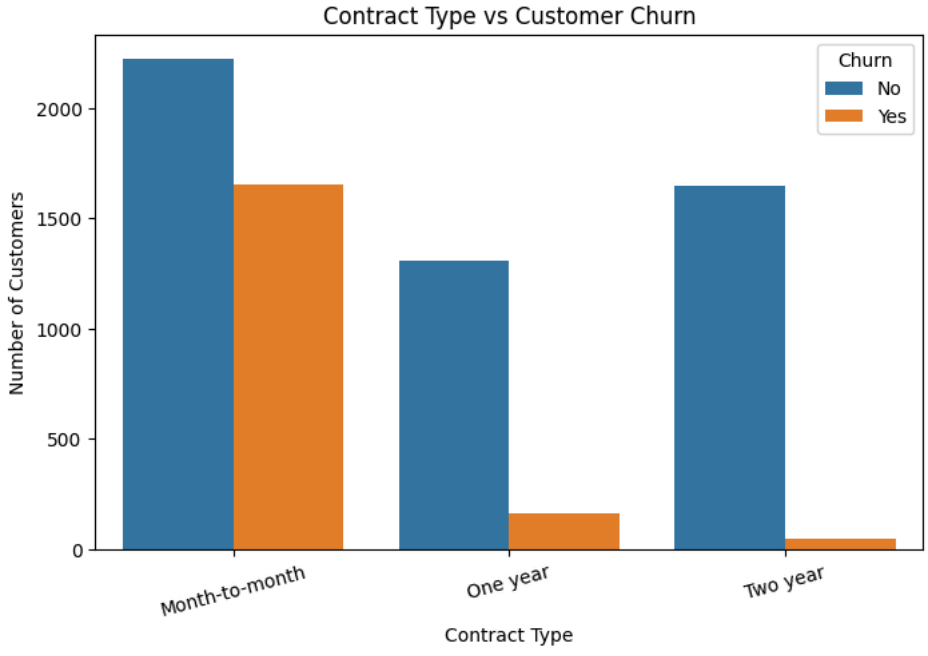
```
plt.xlabel("Tenure (Months)")
plt.ylabel("Frequency")
plt.show()
```




```
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="Contract", hue="Churn")

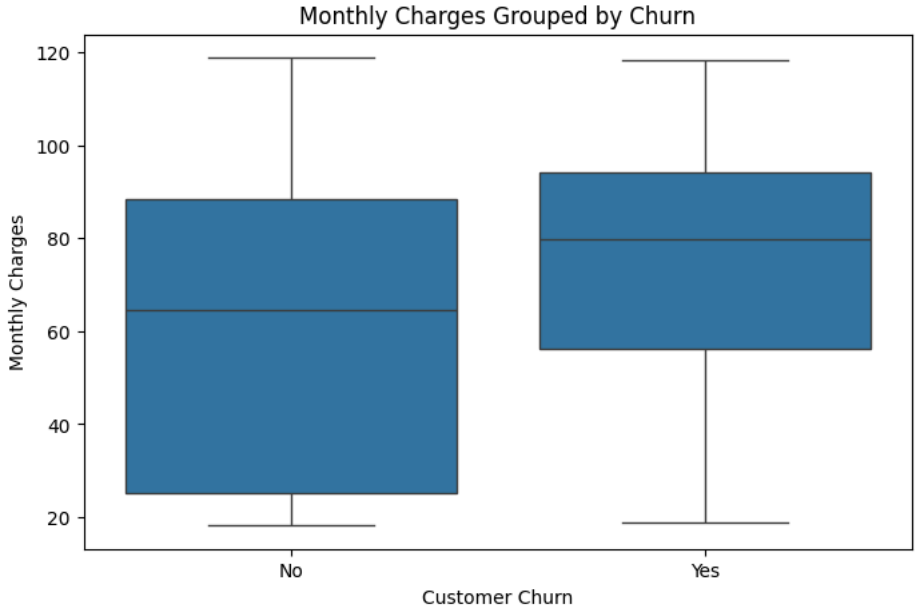
plt.title("Contract Type vs Customer Churn")
plt.xlabel("Contract Type")
plt.ylabel("Number of Customers")
plt.xticks(rotation=15)
plt.show()
```




```
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Churn", y="MonthlyCharges")

plt.title("Monthly Charges Grouped by Churn")
plt.xlabel("Customer Churn")
plt.ylabel("Monthly Charges")
plt.show()
```

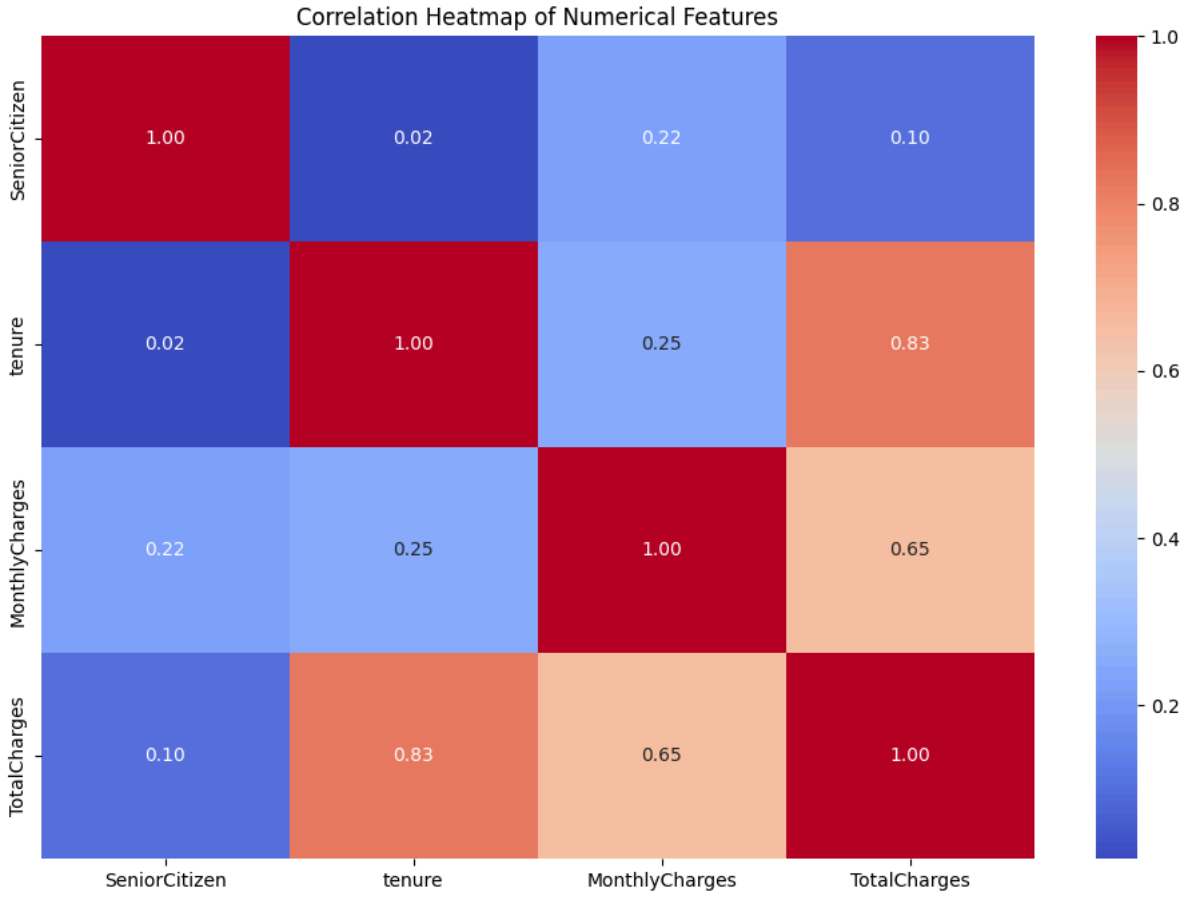



```
numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(12, 8))

sns.heatmap(
    numeric_df.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap of Numerical Features")
plt.show()
```




```
df.to_csv("CustomerChurn_Cleaned.csv", index=False)

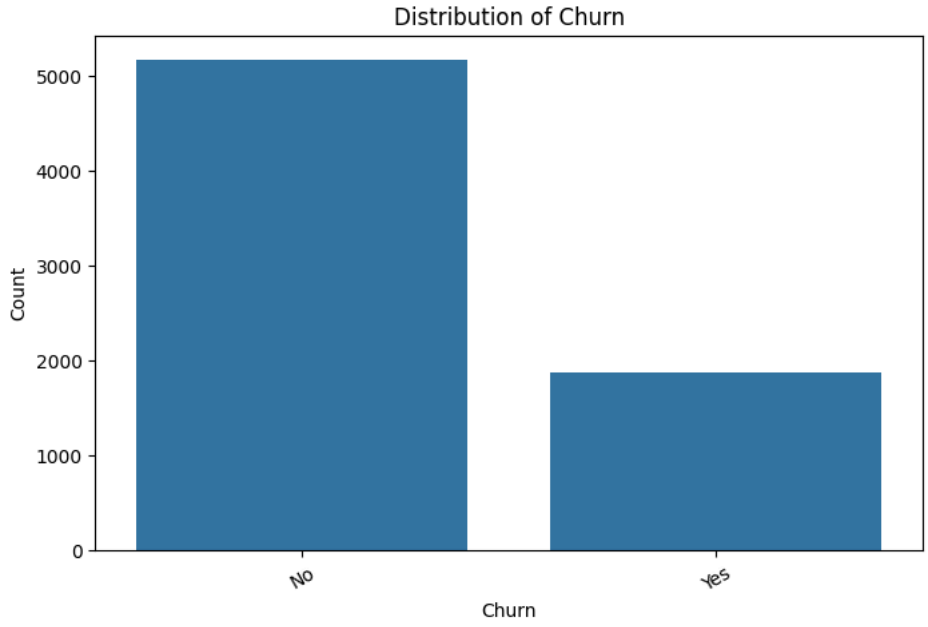
files.download("CustomerChurn_Cleaned.csv")
```

```
columns = [
    "Churn",
    "gender",
    "Contract",
    "PaymentMethod"
]

for col in columns:
    plt.figure(figsize=(8, 5))

    sns.countplot(data=df, x=col)

    plt.title("Distribution of " + col)
    plt.xlabel(col)
    plt.ylabel("Count")
    plt.xticks(rotation=30)
    plt.show()
```



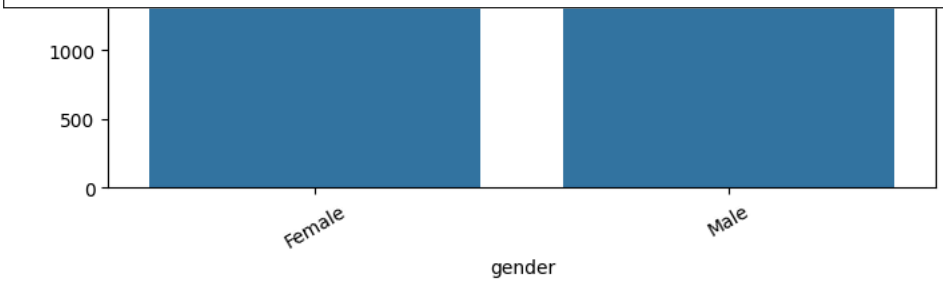

```
columns = ["MonthlyCharges", "TotalCharges", "tenure"]

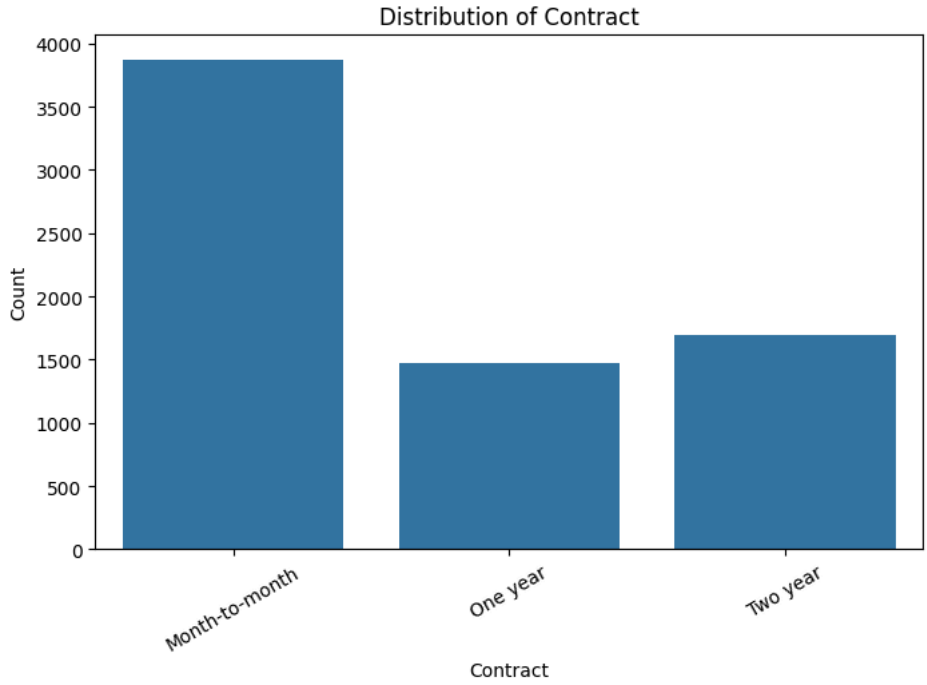
for col in columns:
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))

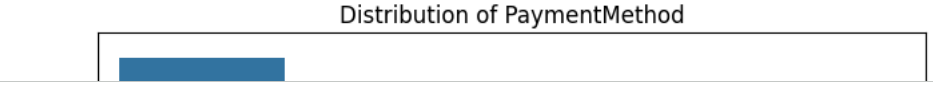
    sns.histplot(df[col], kde=True, ax=ax[0])
    ax[0].set_title(col + " Histogram")

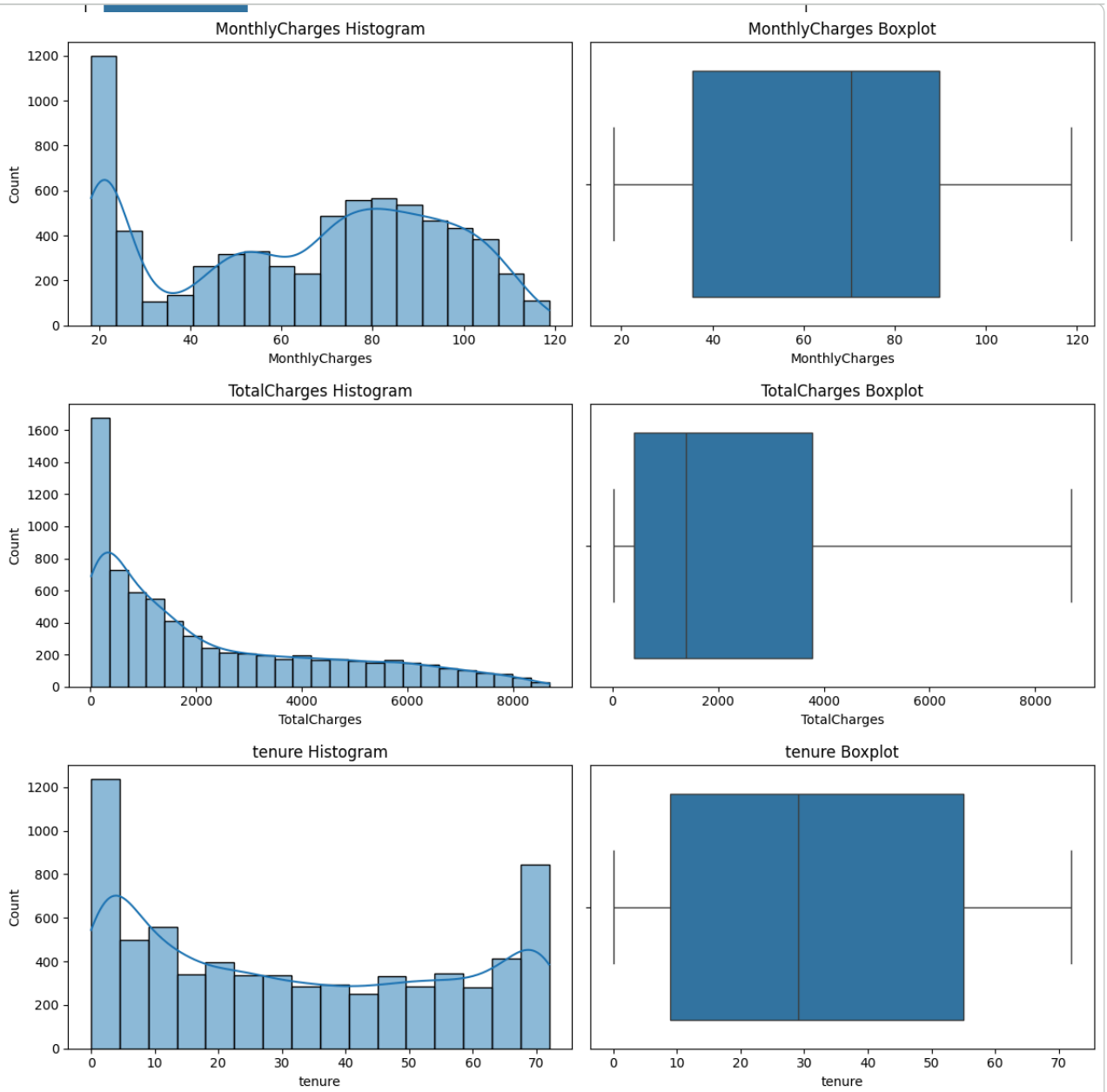
    sns.boxplot(x=df[col], ax=ax[1])
    ax[1].set_title(col + " Boxplot")

    plt.tight_layout()
    plt.show()
```










```
numeric_df = df.select_dtypes(include=np.number)

for col in numeric_df.columns:
    Q1 = numeric_df[col].quantile(0.25)
    Q3 = numeric_df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outliers = numeric_df[
        (numeric_df[col] < lower) |
        (numeric_df[col] > upper)
    ]

    print(
        col,
        "- Number of outliers:",
```

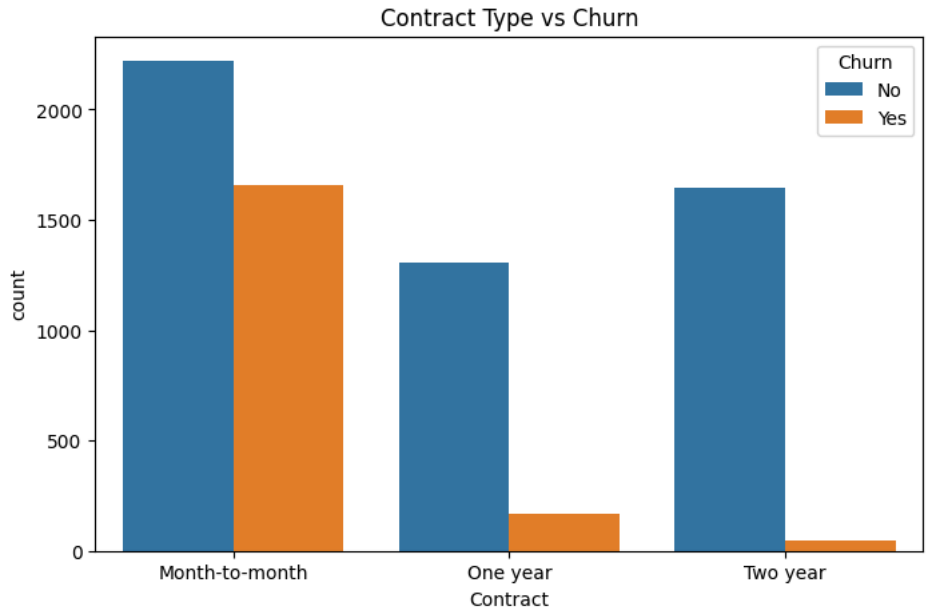
```
        len(outliers),
        "| Lower bound:", round(lower, 2),
        "| Upper bound:", round(upper, 2)
    )
```

```
SeniorCitizen - Number of outliers: 1142 | Lower bound: 0.0 | Upper bound: 0.0
tenure - Number of outliers: 0 | Lower bound: -60.0 | Upper bound: 124.0
MonthlyCharges - Number of outliers: 0 | Lower bound: -46.02 | Upper bound: 171.38
TotalCharges - Number of outliers: 0 | Lower bound: -4674.34 | Upper bound: 8863.16
```

```
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="Contract", hue="Churn")

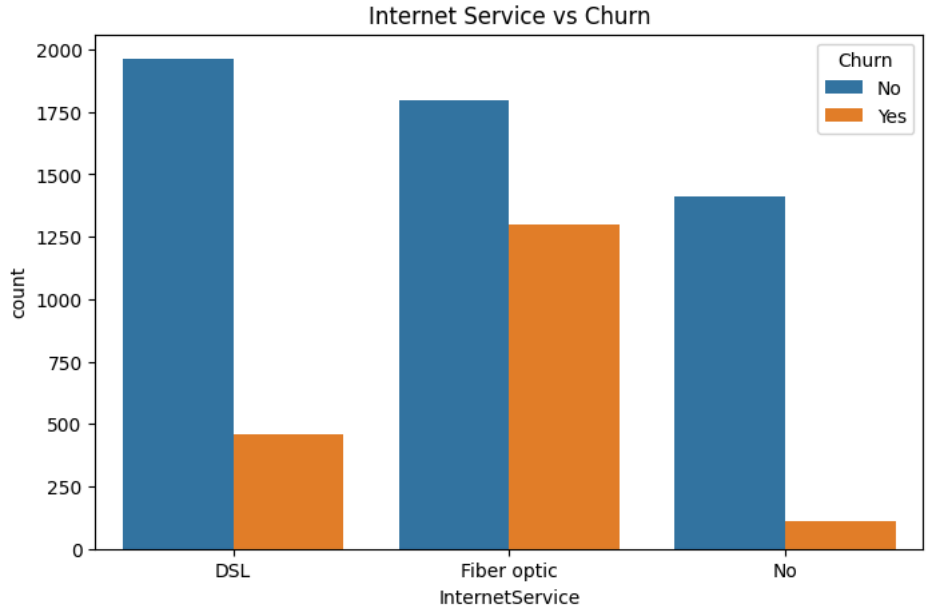
plt.title("Contract Type vs Churn")
plt.show()
```




```
plt.figure(figsize=(8, 5))

sns.countplot(data=df, x="InternetService", hue="Churn")

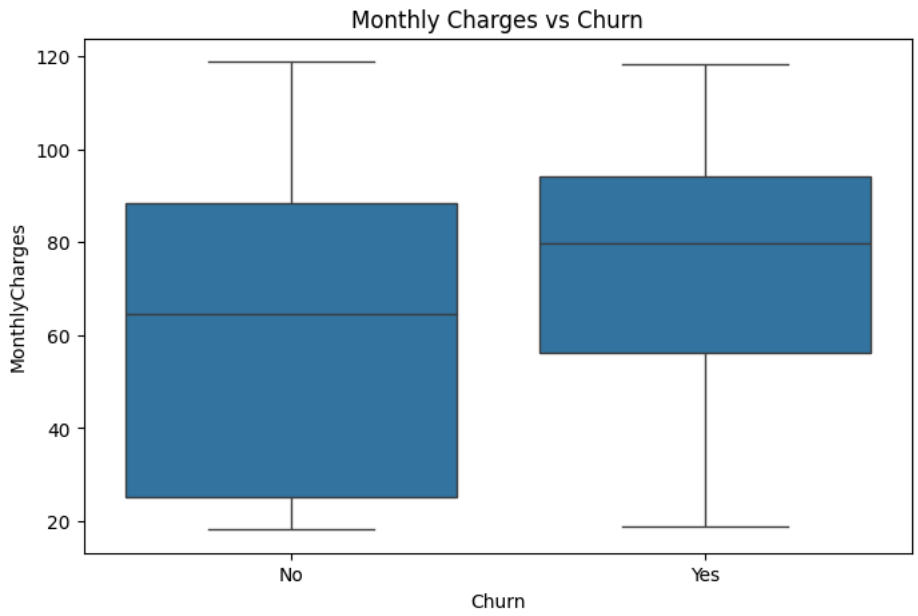
plt.title("Internet Service vs Churn")
plt.show()
```




```
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Churn", y="MonthlyCharges")
```

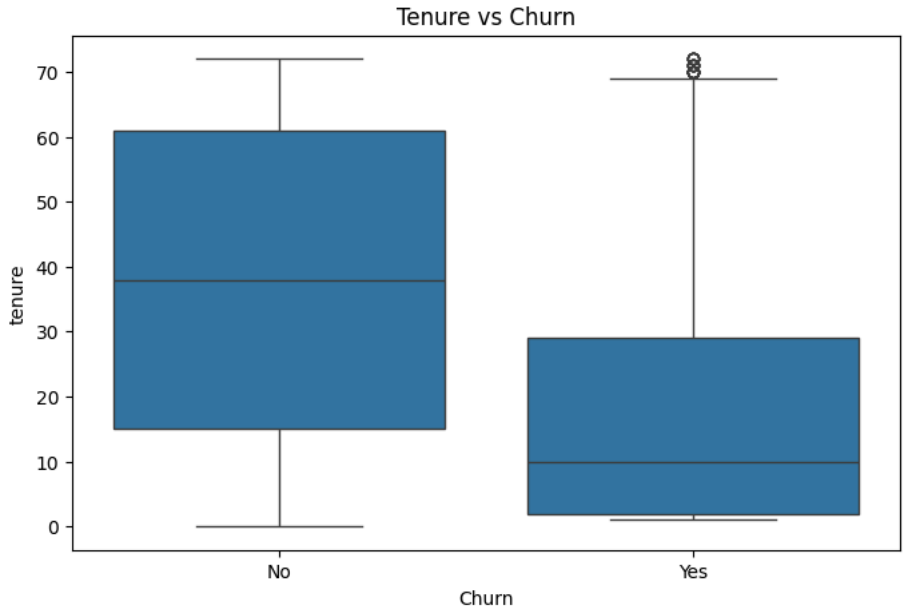
```
plt.title("Monthly Charges vs Churn")
plt.show()
```




```
plt.figure(figsize=(8, 5))

sns.boxplot(data=df, x="Churn", y="tenure")

plt.title("Tenure vs Churn")
plt.show()
```




```
print("1. Churn rate by contract type:")
print(
    df.groupby("Contract")["Churn"]
    .apply(lambda x: (x == "Yes").mean())
    .sort_values(ascending=False)
)

print("\n2. Most frequent payment method among churned customers:")
print(df[df["Churn"] == "Yes"]["PaymentMethod"].value_counts())

print("\n3. Average tenure by churn:")
print(df.groupby("Churn")["tenure"].mean())

print("\n4. Average monthly charges by churn:")
print(df.groupby("Churn")["MonthlyCharges"].mean())

print("\n5. Churn rate by Internet Service:")
print(
    df.groupby("InternetService")["Churn"]
```

```
    .apply(lambda x: (x == "Yes").mean())
    .sort_values(ascending=False)
)

print("\n6. Monthly Charges summary by churn:")
display(
    df.groupby("Churn")["MonthlyCharges"]
    .describe()
)
```

```
1. Churn rate by contract type:
Contract
Month-to-month    0.427097
One year          0.112695
Two year          0.028319
Name: Churn, dtype: float64

2. Most frequent payment method among churned customers:
PaymentMethod
Electronic check             1071
Mailed check                  308
Bank transfer (automatic)     258
Credit card (automatic)       232
Name: count, dtype: int64

3. Average tenure by churn:
Churn
No     37.569965
Yes    17.979133
Name: tenure, dtype: float64

4. Average monthly charges by churn:
Churn
No     61.265124
Yes    74.441332
Name: MonthlyCharges, dtype: float64

5. Churn rate by Internet Service:
InternetService
Fiber optic    0.418928
DSL            0.189591
No             0.074050
Name: Churn, dtype: float64

6. Monthly Charges summary by churn:
```

Churn	count	mean	std	min	25%	50%	75%	max
No	5174.0	61.265124	31.092648	18.25	25.10	64.425	88.4	118.75
Yes	1869.0	74.441332	24.666053	18.85	56.15	79.650	94.2	118.35

```
from sklearn.model_selection import train_test_split

# Remove customer ID and separate the target
X = df.drop(columns=["customerID", "Churn"])
y = df["Churn"].map({"No": 0, "Yes": 1})

# Convert categorical columns into numerical columns
X = pd.get_dummies(X, drop_first=True, dtype=int)

# Split into training and testing datasets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training features:", X_train.shape)
print("Testing features:", X_test.shape)

# Save the complete model-ready dataset
processed_df = X.copy()
processed_df["Churn"] = y

processed_df.to_csv(
    "CustomerChurn_Preprocessed.csv",
    index=False
)

files.download("CustomerChurn_Preprocessed.csv")
```

```
Training features: (5634, 30)
Testing features: (1409, 30)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

models = {
    "Logistic Regression": make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=2000)
    ),

    "Decision Tree": DecisionTreeClassifier(
        random_state=42
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
}

results = []

for name, model in models.items():
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, zero_division=0
        ),
        "Recall": recall_score(
            y_test, y_pred, zero_division=0
        ),
        "F1-Score": f1_score(
            y_test, y_pred, zero_division=0
        )
    })

    print("\n", name)
    print(classification_report(
        y_test, y_pred, zero_division=0
    ))

    cm = confusion_matrix(y_test, y_pred)

    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
    plt.title("Confusion Matrix - " + name)
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.show()
```

```
Logistic Regression
              precision    recall  f1-score   support

           0       0.85      0.89      0.87      1035
           1       0.66      0.57      0.61       374

    accuracy                           0.81      1409
```

```
results_df = pd.DataFrame(results).set_index("Model")

display(results_df)

results_df.plot(
    kind="bar",
    figsize=(10, 6),
    ylim=(0, 1)
)

plt.title("Classification Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.806955	0.658385	0.566845	0.609195
Decision Tree	0.741661	0.513889	0.494652	0.504087
Random Forest	0.786373	0.623729	0.491979	0.550075

